FUNDACIÓN UNIVERSITARIA AGRARIA DE COLOMBIA – UNIAGRARIA
Dirección de Educación Virtual · Ciencia de Datos (Profesional)
Actividad 2 – Hands on: configuración de herramientas
Estudiante: Andres Felipe Amaya Camacho
Docente: Carlos Andres Gomez Vasco

Capítulo 3. Ramas en Git

Crear, fusionar y gestionar ramas; flujos de trabajo y rebase

Las ramas permiten trabajar en líneas de desarrollo independientes sin afectar la versión principal. En Git son tan baratas que se crean y descartan constantemente; este capítulo explica por qué y cómo usarlas.

3.1 Ramas en pocas palabras

Una rama es un puntero móvil a un commit. La rama por defecto suele llamarse `main` (antes `master`) y `HEAD` es el puntero especial que indica en qué rama se está trabajando. Como una rama es solo un archivo con 40 caracteres (el hash del commit), crearla es instantáneo.

```
git branch nueva-rama            # crea la rama
git switch nueva-rama            # cambia a ella
git switch -c nueva-rama         # crea y cambia
git checkout -b nueva-rama       # forma clásica equivalente
git log --oneline --graph --all  # visualizar ramas
```

3.2 Procedimientos básicos de ramificación y fusión

Un caso típico: se crea una rama para una corrección (hotfix) o una funcionalidad, se confirma allí, y al terminar se fusiona con `main`. Si `main` no avanzó, Git hace una fusión *fast-forward* (solo mueve el puntero). Si ambas ramas avanzaron, crea un commit de fusión con dos padres.

```
git switch main
git merge nueva-rama
git branch -d nueva-rama   # elimina la rama ya fusionada
```

Conflictos: si las dos ramas modificaron la misma parte de un archivo, Git pausa la fusión y marca el conflicto con `<<<<<<<`, `=======` y `>>>>>>>`. Se edita el archivo, se deja la versión correcta y se completa con `git add` y `git commit`. `git mergetool` abre una herramienta gráfica para ayudar.

3.3 Gestión de ramas

```
git branch                 # lista las ramas locales
git branch -v              # último commit de cada una
git branch --merged        # ya fusionadas con la actual
git branch --no-merged     # con trabajo sin fusionar
git branch -D rama         # fuerza el borrado
```

```
git branch -m viejo nuevo  # renombrar
```

3.4 Flujos de trabajo con ramas

- **Ramas de larga duración:** `main` solo con código estable; `develop` o `pruebas` para integración.
- **Ramas temáticas:** de vida corta, una por tarea o error (por ejemplo `iss53`). Facilitan revisar y descartar trabajo.

3.5 Ramas remotas

Las ramas remotas (`origin/main`) son referencias locales que recuerdan el estado del remoto en la última sincronización. Se actualizan con `git fetch`. Una rama local puede *rastrear* una remota para que `git pull` y `git push` sepan a dónde ir.

```
git push origin mi-rama
git switch -c mi-rama origin/mi-rama
git branch -u origin/mi-rama
git push origin --delete mi-rama
```

3.6 Reorganizar el trabajo (rebase)

`git rebase` toma los cambios de una rama y los vuelve a aplicar sobre otra, dejando una historia lineal más limpia que la de una fusión. La regla de oro: **no hacer rebase de commits que ya se publicaron** y que otras personas pueden haber usado, porque se reescribe la historia.

```
git switch tema
git rebase main
git switch main
git merge tema      # ahora es un fast-forward
```

Conclusión del capítulo

Ramificar es barato y seguro: permite experimentar sin riesgo y organizar el trabajo en equipo. Elegir entre merge y rebase depende de si se prefiere conservar la historia tal cual o mantenerla lineal.

Fuente: Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2